Template matching

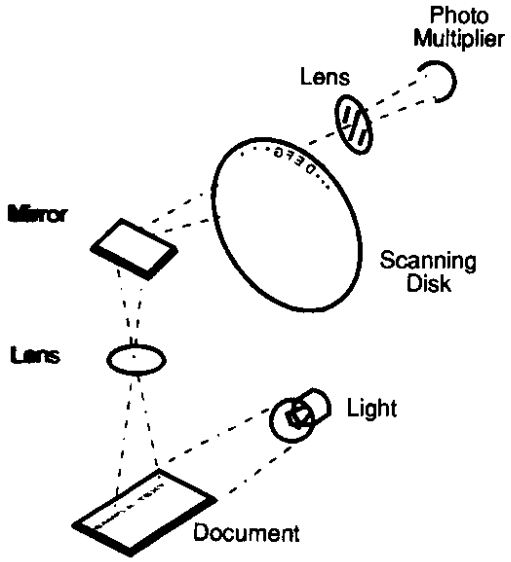
Each class is represented by one template (prototype).

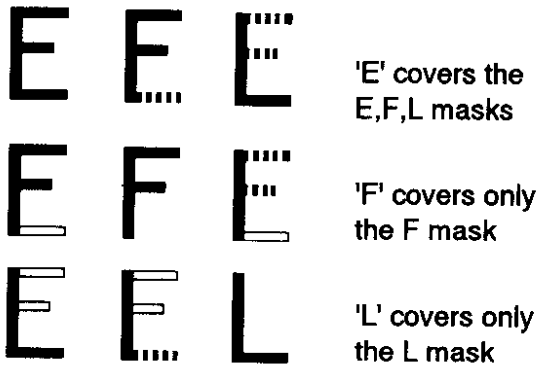
It is assumed that classified objects are templates with addition of noise.

We compare object to be classified with each template seeking:

- Maximum correlation
- Minimum error

Historically it was the first approach to pattern classification.





Minimum-distance classification

Let us replace *template* by its feature vector. Let the index of the template be class label.

Instead of templates themselves we have for each class its feature vector: t^1, t^2 ... t^L (where L is number of classes in our problem).

When we want to classify an object we should first compute its feature vector. Let us denote it by *x*.

In the next step we compute L distances between *x* and each template t^i: $d^i = d(x, t^i)$

The final step is to find minimum distance between x and class templates:

$$c = \arg\min_i d^i$$

c is classification result (class label).

Distance function (1)

Distance function must satisfy 4 axioms:

1. $d(x,y) = 0 \equiv x = y$ (reflexivity)
2. $d(x,y) \geq 0$ (distances are non-negative)
3. $d(x,y) = d(y,x)$ (symmetry)
4. $d(x,y) + d(y,z) \geq d(x,z)$ (triangle inequality)

Distance functions (2)

Hamming distance

$$d_H(x,y) = \sum_{i=1}^{d} x_i \oplus y_i$$

City-block distance (Manhattan)

$$d_m(x,y) = \sum_{i=1}^{d} |x_i - y_i|$$

Euclidean distance

$$d_E(x,y) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

Minkowski metric

$$d_m(x,y) = \left[\sum_{i=1}^{d} |x_i - y_i|^s \right]^{1/s}$$

Levenshtein (edit) distance

Nearest neighbour classification

Training set

$$T = \{ \langle x^k, y^k \rangle, k=1,2, \ldots N \}$$

x^k k^{th} object feature vector,
y^k k^{th} object class label

N – size of the training set

Nearest neighbour of x:

$$x^m \in T: \min d(x, x^k) = d(x,x^m) \text{ for } k=1,2, \ldots N$$

We decide, that x belongs to the same class as y^m - its nearest neighbour.

This classifier is abbreviated 1-NN, because only one nearest neighbour decides about the classification result.

1-NN Properties

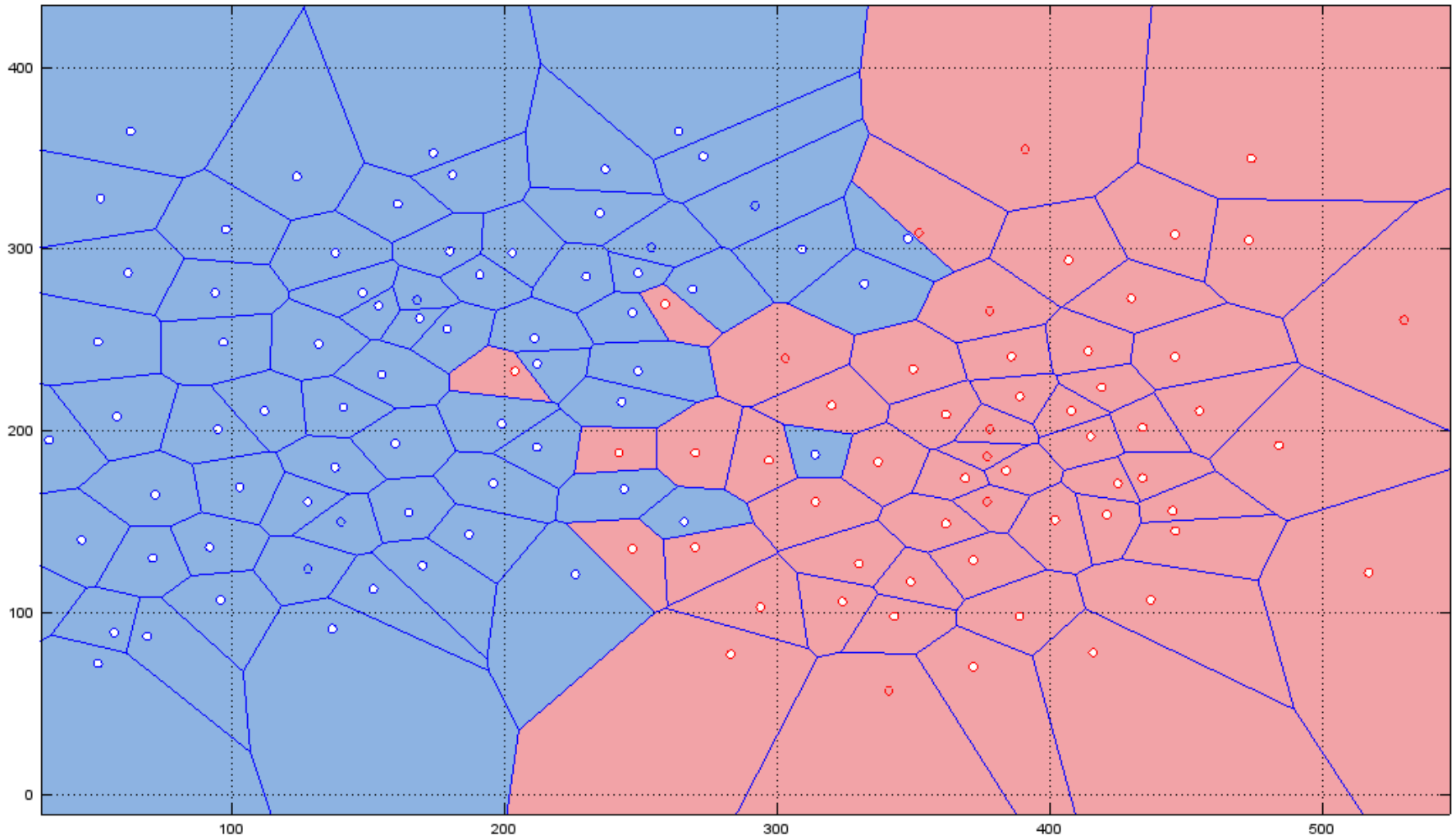
Advantages of 1-NN classification:

- Direct use of the training set
- Low probability of error
- Simple implementation

1-NN classification shortcomings:

- Large computational complexity for large N
- Sensitivity to misclassified samples
- "random" behaviour at class boundaries

Voronoi diagram



k-NN classification

Majority voting of *k* nearest neighbours.

Set of *k* neighbours:

$$T^k = \{ \langle x^\nu, y^\nu \rangle, \nu=1,2, \ldots k \}$$

Subset of T^k containing elements of class y:

$$T^{k,y} = \{ \langle x^\nu, y^\nu \rangle, \nu=1,2, \ldots k \wedge y^\nu=y \}$$

Membership function:

$$F^y(x) = \# T^{k,y} \quad y=1, 2, \ldots L$$

Decision: x belongs to class y^m where $F^{ym}(x) = \max F^y(x)$

k selection for k-NN classification

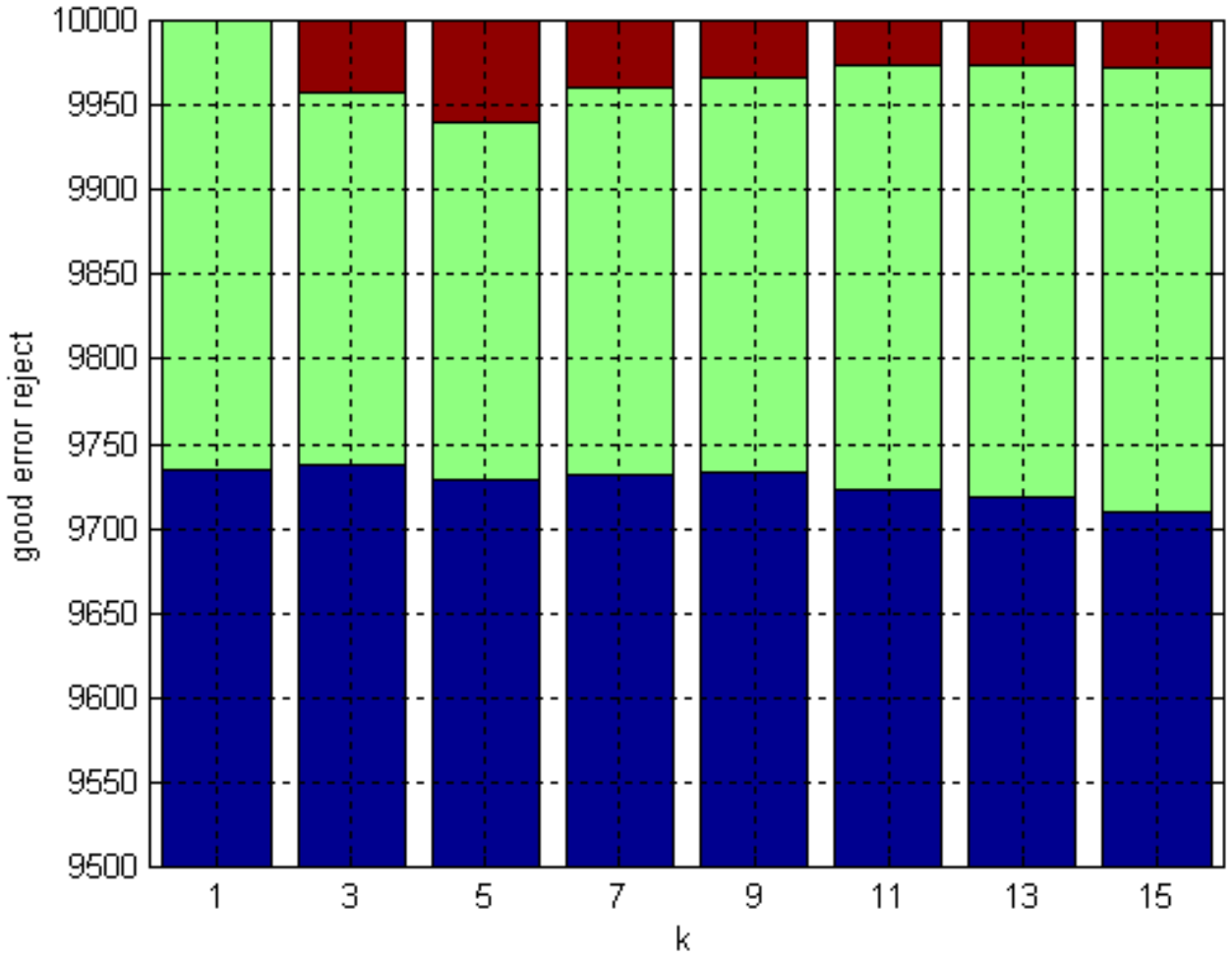
We have two contradictory requirements for k:

- large, to minimize probability of misclassification,
- small - with relation to training set size - to achieve true class boundaries.

For a fixed size training set we should experiment with different k values and select the value giving best classification results on test set.

Generally, we will observe the shift of the errors into the reject decision; it is desirable when the reject option has lower cost than error in classification. Next figure shows the classification coefficients for different *k*s. (Handwritten digits were classified in 40 dimensions using Euclidean distance).

What *k* value should we select?



Data preprocessing

In nearest neighbor classifier it is evident that all features should have the same order of magnitude – but it's easy to forget it.

The first step in designing classifier (any classifier! not just NN) is learning the data. The goal is twofold:

1. To detect outliers (if any)
2. To make decision if normalization should be performed

Usually some simple checking will do:

1. Compute mean and variance of features
2. Compare mean and median of features
3. Check minimum and maximum features' values
4. Plotting histogram of individual features

Data normalization

In many cases it is enough to scale the input data. We divide feature values in most cases by:

Square root of variance

Total feature values range (max – min)

These parameters must be computed once – and on the training set.

If you normalize the training set you have to normalize in exactly the same way data to be classified.

NN classifier quality assessment

Cross-validation procedure:

- Data set is partitioned into K subsets
- Each of the K subsets is used as the testing set, while remaining subsets form the training set
- The estimate of our NN classifier recognition (or error) coefficient is the mean value from our K tests.

Stratification – we should pay attention to number of samples belonging to different classes in each of K subsets.

Leave-one-out method is the “extreme” cross-validation: the test set contains just one sample!

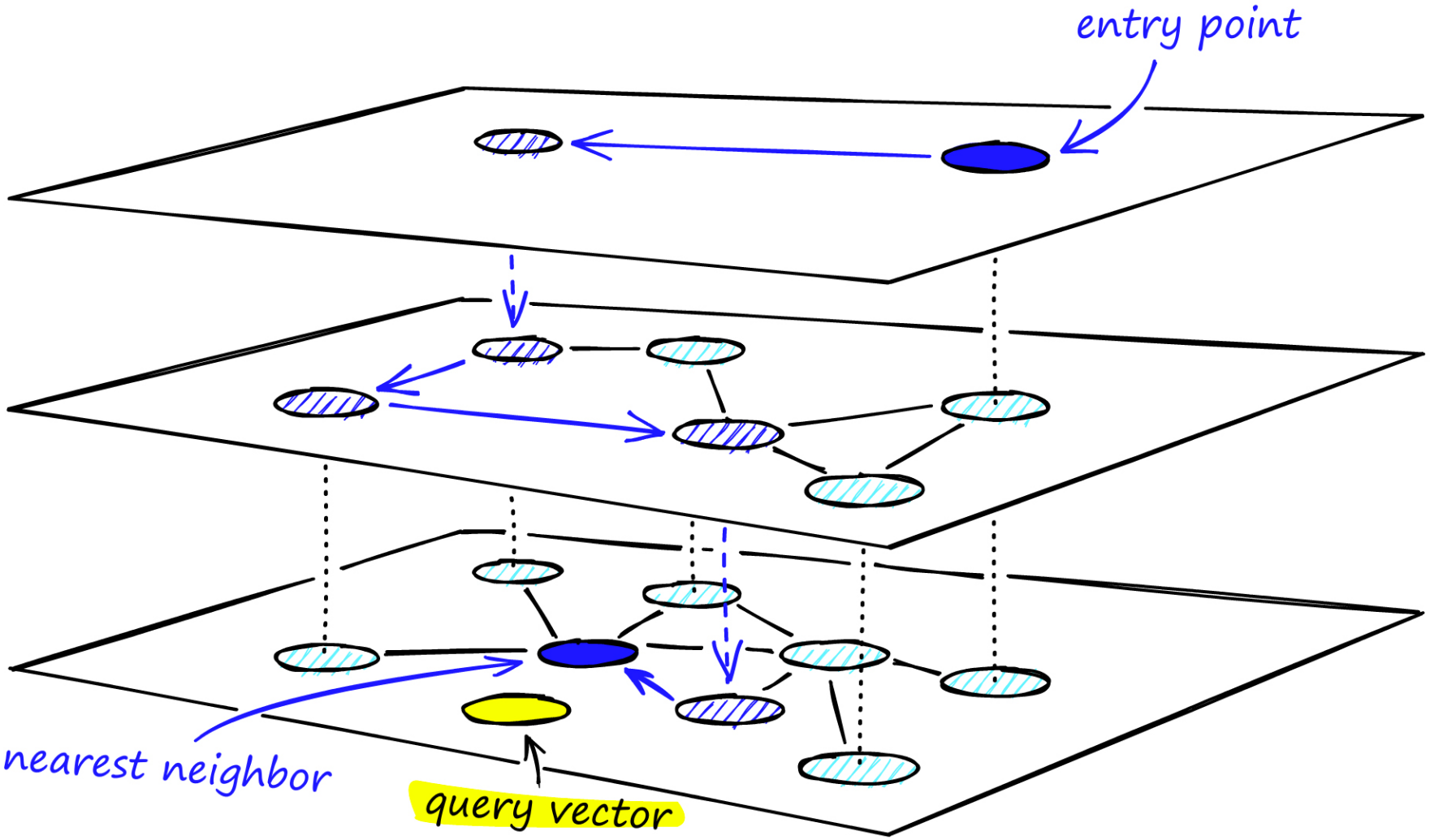
Approximate Nearest Neighbour Search

Sometimes it makes sense to have slightly imprecise results if we can get them quickly.
Imagine that we selected k = 15 and use majority voting when making classification decision. In this situation one or even two samples from the training set replacing “true” nearest neighbours will probably not change the classification outcome.

There are many interesting approaches to such an imprecise search. A good starting point to learn them (with real implementation discussed) is:
Faiss: The Missing Manual, James Briggs (Pinecone)
https://www.pinecone.io/learn/series/faiss/ (checked on 2.03.2026)

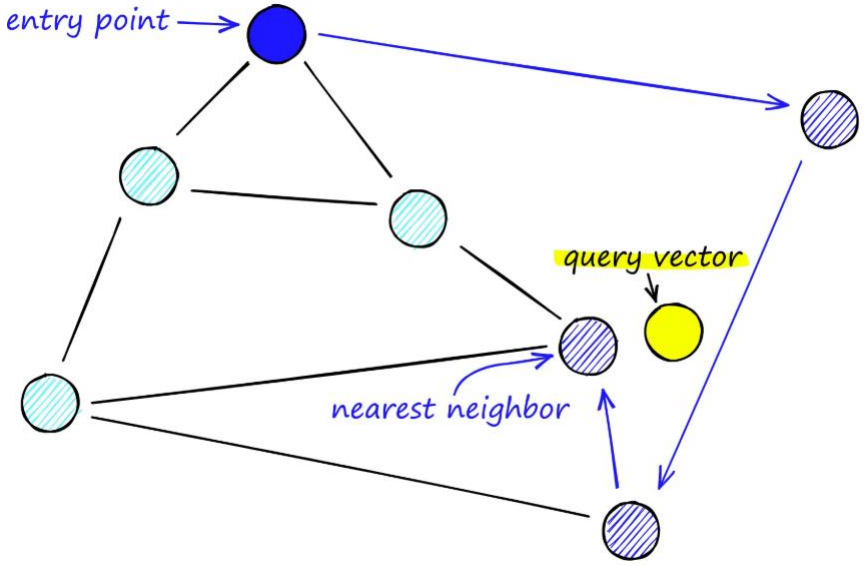
Faiss == Facebook AI Similarity Search

Hierarchical Navigable Small Worlds (HNSW)



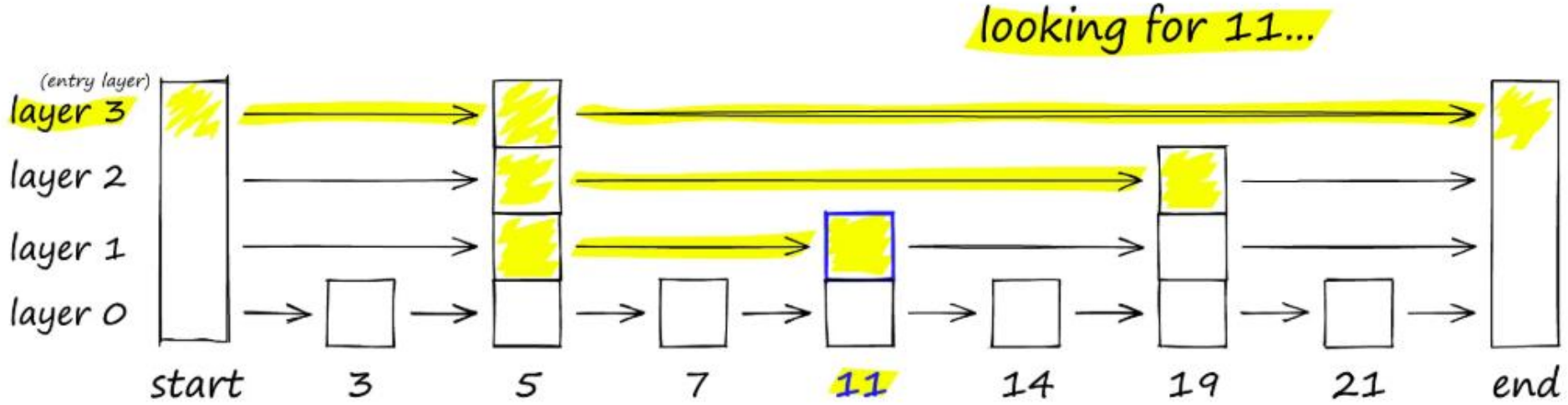
Search in HNSW – just one layer at a time

We start with fixed entry point and compute distances of its neighbours to the query vector. We select the nearest one as current point.
We repeat with the current point the procedure of computing the distances and selecting current point.
We end when there are no neighbours nearer the query vector than current point (current point becomes our ANN candidate)



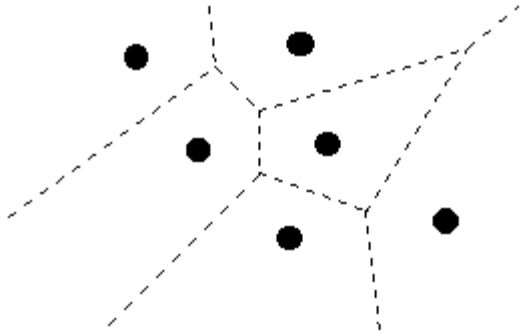
Search in HNSW – layers analogy

Hierarchy can easily be illustrated with so called skip lists. Note that on the image below top layer contains very few values (points), while the bottom layer contains all the points in our list.
Note also that we move down the layer structure when we learn that next point in current layer is greater than our search value.

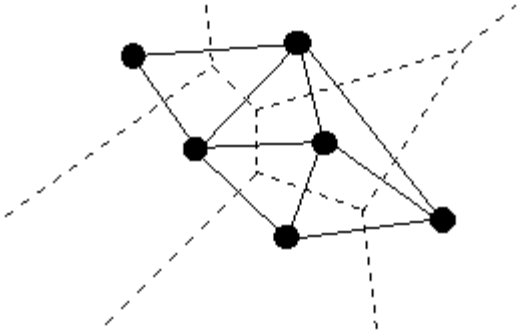


Condensation of the training set

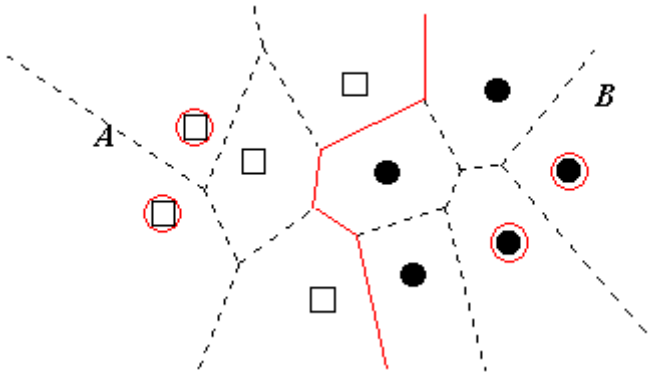
Voronoi diagram - for given set of nodes it is space division into regions, in which all points are closer to region's node than to any other node.



Delaunay triangulation - it's dual of Voronoi diagram. If two Voronoi regions have common edge, then nodes of these regions are connected. Such nodes are called Voronoi (Delaunay) neighbours.



Decision boundary for 1-NN classification belongs to Voronoi diagram.



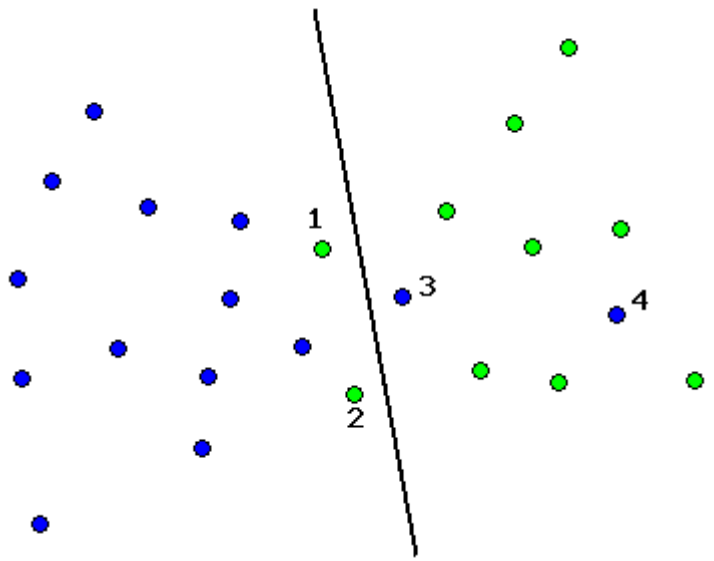
Voronoi regions' nodes, which edges do not belong to decision boundary are superfluous. They can be removed from training set.

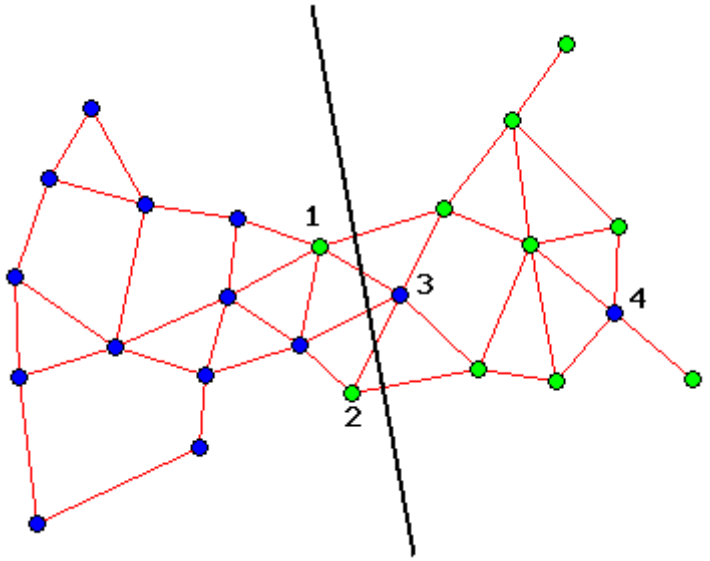
Condensing algorithm

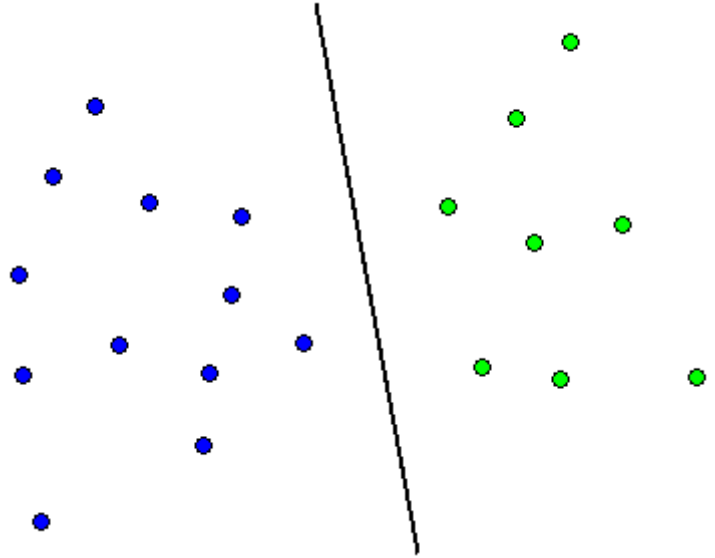
1. Compute Delaunay triangulation for training set.
2. For each node, mark if all its Delaunay neighbours belong to the same class as the node considered.
3. Remove all *marked* nodes. The remaining nodes comprise condensed training set.

Editing algorithm

1. Build proximity graph of the training set.
2. For each graph's node:
   a. Find all neighbours of the node.
   b. Determine most frequent class in the neighbourhood (winning class)
   c. *Mark* if node belongs to class other than neighbourhood's winning class.
3. Remove *marked* nodes from the training set.


1
2
3
4


1
2
3
4



Training set reduction - MULTIEDIT

Devijver and Kittler (1982)

1. Diffusion
   Split randomly training set T into M subsets T_1, ... T_M, $M \geq 3$.
2. Classification
   Classify samples from set T_i using 1-NN method with $T_{(i+1) \bmod M}$ as the training set, i = 1, ... M.
3. Editing
   Remove all samples misclassified in step 2.
4. Blending
   Join remaining samples into new training set T.
5. Termination
   If M last iterations did not change set T terminate, in other cases return to step 1.

Condensing algorithm

We set up bins called Store and Grabbag. The first sample is placed in Store all the other samples are placed in Grabbag. We let n_k designate number of samples in Grabbag whenever step 1 of the algorithm is entered.

1. For i = 1, ... n_k, that is all samples in Grabbag
   Classify *i*th sample of Grabbag with 1-NN rule using Store as the training set. If classified *correctly* the sample is returned to Grabbag, otherwise it is placed in Store.
2. If no sample was transferred from Grabbag to Store in step 1, or Grabbag is empty then terminate; else return to step 1.

The final contents of Store constitute the condensed training set for 1-NN classifier.